

MINI PROJECT – PARALLEL MANDELBROT PROGRAM

Link to repository: https://github.com/tishayne/concurrency_project.git

For this project, I used the file **Mandel4.java** from **java-rubikscube.info** as a starting point and developed my own version from there. The original program already displayed the Mandelbrot set in a Java Swing window. The final project had to use parallel execution and run as a normal program.

In the Mandelbrot set, every pixel can mostly be calculated independently. However, the calculation can be expensive. Some pixels are finished very quickly, while others need many iterations. Therefore, the program is a good case for testing how multiple threads can improve performance.

From work, I have seen the use of thread pools, and I think they are commonly used in industry when many tasks have to run in parallel. A thread pool avoids creating and destroying threads manually. Because of this, I used a fixed thread pool for the renderer.

First version

In my first parallel version, I divided the image into blocks of rows. Each thread received one part of the image and calculated its pixels. This already improved the program compared to a sequential version, because several CPU cores could work at the same time.

However, there was one important problem. The Mandelbrot image is not equally difficult everywhere. Some rows are easy and are calculated quickly. Other rows are much slower because the pixels need many iterations. This means that one thread can finish early, while another thread is still working on a more complex row.

The total render time is not only about the number of threads, but also about how fairly the work is distributed. In the first version, the program could have idle threads while another thread still had a lot of work. This is a load balancing problem.

Second version - Load Balancing

To improve this, I changed the work distribution. Instead of giving a fixed block of rows to each thread, I used an `AtomicInteger` as a small ticket machine.

The counter starts at row 0. Each worker thread takes the next available row, renders it, and then asks for another row. When all rows are finished, the workers stop. This means that threads take new work dynamically. If one thread gets a difficult row, the other threads can still continue with other rows. This keeps the CPU cores busier.

As another approach, I remembered the work-stealing lecture and thought that this could also be a possibility. However, it would have been too complex for this project, so I chose the easier ticket machine approach.

In the first version, the program waited for the slowest block of rows. In this version, the program usually waits only for the last remaining row. This gives better load balancing.

Third version

Render Settings & RenderResult

After that, I tried to structure the program into clearer classes. I created immutable `RenderSettings` and `RenderResult` objects.

RenderSettings stores the current settings for rendering. This makes the program easier to understand, because each render operation gets a fixed snapshot of the settings. Because the fields are final, the same settings object can safely be used by several worker threads.

Redraw and Stop Handling

The Swing user interface must stay responsive. Therefore, the rendering should not run directly on the Swing event dispatch thread. I used a separate render thread for the rendering process.

When the user changes something, for example the zoom, palette, iteration count, antialiasing, or thread count, the panel calls redraw(). The flag redrawRequested tells the render thread that a new image is needed. The render thread waits while there is no work. When redraw() is called, notifyAll() wakes it up.

I also fixed the handling of old renders. If the user zooms while an image is still rendering, the old image is no longer useful. In this case, the render thread is interrupted. The renderer checks whether the current thread was interrupted. If yes, it stops the old render and starts again with the newest settings.

Additionally, I added a stopped flag, which gives the render thread a clean way to exit when the panel is closed.

Fourth version - Reusing ExecutorService, unsafe BufferedImage, MandelbrotPanel

In this version I made three improvements.

First, the ExecutorService is now reused across renders and only recreated when the thread count changes. In the previous version, a new thread pool was created and destroyed on every render. This added unnecessary overhead, especially during interactive use where renders happen frequently.

Second, I changed the rendering to write pixels into a plain int[] array instead of calling setRGB directly on the BufferedImage from multiple threads. BufferedImage is not thread-safe. Even though each thread writes to a different row (via the ticketing machine), the internal WritableRaster is shared, and the Java specification does not guarantee that concurrent writes are safe. It works in practice, but it is technically undefined behavior. Writing into an int[] array first and then setting the image pixels once at the end removes this problem and makes it thread-safe.

I also split MandelbrotPanel into smaller parts. The render loop logic was extracted into a separate inner class, and the input handlers were separated. This makes the class easier to read and maintain.

How to run the program

The application is started from MandelbrotApplication.java. It creates the JFrame, adds the MandelbrotPanel, shows the window, requests keyboard focus, and starts the first render. This and all keyboard shortcuts are also explained in the README.md file.

Running Benchmark

The benchmark is started by running the application and pressing the **B** key. It will print the average render time with the current render settings. To change the number of iterations, just adjust the render settings. Then the **B** key must be pressed again.

The benchmark runs in a separate thread. It renders the same Mandelbrot view multiple times with one thread and a selected number of threads. The average render time is calculated and printed to the console as shown here.

```
==== Mandelbrot Benchmark ====
Average render time with 1 thread: 131 ms
Average render time with 4 threads: 31 ms
Speedup: 4.23x
Iterations: 192
|
```

1: Console output Benchmark

This feature was created only for the assignment, so I did not program a separate frame for it. I wanted to keep it simple and add as little code as possible. That is why the result is only printed in the console and the number of threads is hard-coded. For the assignment, the important comparison is one thread against four threads. I tested several thread counts and iteration counts.

Max iterations [C]	1 thread ms	2 threads ms	4 threads ms	8 threads ms
192	131	39	31	22
512	180	83	44	35
1024	332	154	76	53
1600	494	316	114	71
Speedup vs 1 thread	1x	3.36x* 2.13x 2.14x 1.62x	4.23x 4.05x 4.29x 3.99x	5.95x 5.17x 6.09x 7.06x

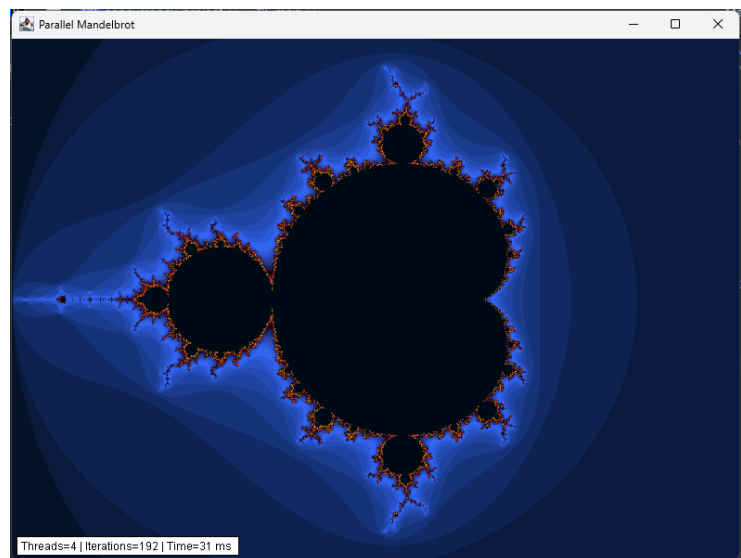
**Example: 39ms x 3.36 = 131ms*

The results show that 4 threads give roughly 4 times the speed of 1 thread at higher iteration counts. At low iteration counts, the speedup is smaller. This is expected. With fewer iterations, each row finishes quickly, so the fixed cost of thread coordination and scheduling becomes a larger part of the total time. This is consistent with Amdahl's Law: the sequential parts of the program, such as image creation and thread management, limit the maximum possible speedup regardless of how many threads are used.

Conclusion

The final version improves the first version in many ways. First, it uses a thread pool and dynamic row scheduling with an AtomicInteger ticket counter, which reduces idle waiting caused by uneven row block complexity. Secondly, it separates redraw requests, render cancellation, and clean thread termination with redrawRequested, interrupt handling, and the stopped flags. Finally, it reuses the ExecutorService across renders and writes pixels into an int[] array instead of directly into BufferedImage, which removes unnecessary thread pool overhead and fixes a thread safety issue in the rendering.

Additionally, with the benchmark experiment I can show that I am able to go 4 times faster with 4 cores than with only 1 core.



2: Final Parallel Mandelbrot Application